

```

import Anthropic from '@anthropic-ai/sdk';
import { CONFIG, PROFILE } from './config.js';

const client = new Anthropic({ apiKey: CONFIG.anthropic.apiKey });

const SYSTEM_PROMPT = `You are a job-match evaluator for a freelance consultant n

Your job: score each incoming gig from 0-10 on how well it fits her profile.

PROFILE:
${PROFILE.summary}

CORE SKILLS:
- AI/Automation: ${PROFILE.skills.automation_ai.join(', ')}
- Backend: ${PROFILE.skills.backend.join(', ')}
- Blockchain: ${PROFILE.skills.blockchain.join(', ')}
- Legal Ops: ${PROFILE.skills.legal_ops.join(', ')}

TARGET RATE: $75-250/hr or $500+ fixed projects

SCORING RULES:
- 9-10: perfect match (AI automation, blockchain, legal ops at good rate)
- 7-8: strong match (adjacent skill at good rate)
- 5-6: possible but weak match
- 0-4: skip

PENALIZE HEAVILY:
- Data entry, pure VA work, transcription
- Rate below $75/hr or budget below $500
- Vague descriptions, no clear scope
- "Urgent 1 hour" or similar red flags
- Equity-only or revenue-share-only

BOOST SCORE for:
- Retainer/monthly/ongoing mentions (+1)
- Claude API, Anthropic, LangChain specifically (+1)
- Legal tech, compliance tech (+1)
- Blockchain + compliance combo (+2)

Return ONLY valid JSON:
{
  "score": 0-10,
  "category": "ai_automation" | "blockchain" | "legal_ops" | "backend" | "other",
  "reasoning": "1-2 sentence explanation",

```

```

    "red_flags": ["list if any"],
    "key_matches": ["skills from profile that match"]
  }`;

export async function evaluateJob(job) {
  const userContent = `JOB TITLE: ${job.title}

BUDGET: ${job.budget_type} ${job.budget_min || '?'}-${job.budget_max || '?'}

DESCRIPTION:
${job.description}`;

  try {
    const res = await client.messages.create({
      model: CONFIG.anthropic.evaluatorModel,
      max_tokens: 500,
      system: SYSTEM_PROMPT,
      messages: [{ role: 'user', content: userContent }],
    });

    const text = res.content[0].text.trim();
    const cleaned = text.replace(/```\json\n?|```n?/g, '').trim();
    const parsed = JSON.parse(cleaned);

    return {
      score: parsed.score || 0,
      category: parsed.category || 'other',
      reasoning: parsed.reasoning || '',
      redFlags: parsed.red_flags || [],
      keyMatches: parsed.key_matches || [],
    };
  } catch (err) {
    console.error(`Avaliação falhou pra ${job.id}:`, err.message);
    return { score: 0, category: 'error', reasoning: err.message, redFlags: [], k
  }
}

export function passesFilter(job, evaluation) {
  if (evaluation.score < CONFIG.filter.minMatchScore) return false;

  const desc = (job.title + ' ' + job.description).toLowerCase();
  for (const bad of CONFIG.filter.blacklistKeywords) {
    if (desc.includes(bad.toLowerCase())) return false;
  }

  if (job.budget_type === 'hourly' && job.budget_min && job.budget_min < CONFIG.f
    return false;

```

```
}  
if (job.budget_type === 'fixed' && job.budget_min && job.budget_min < CONFIG.fi  
    return false;  
}  
  
return true;  
}
```